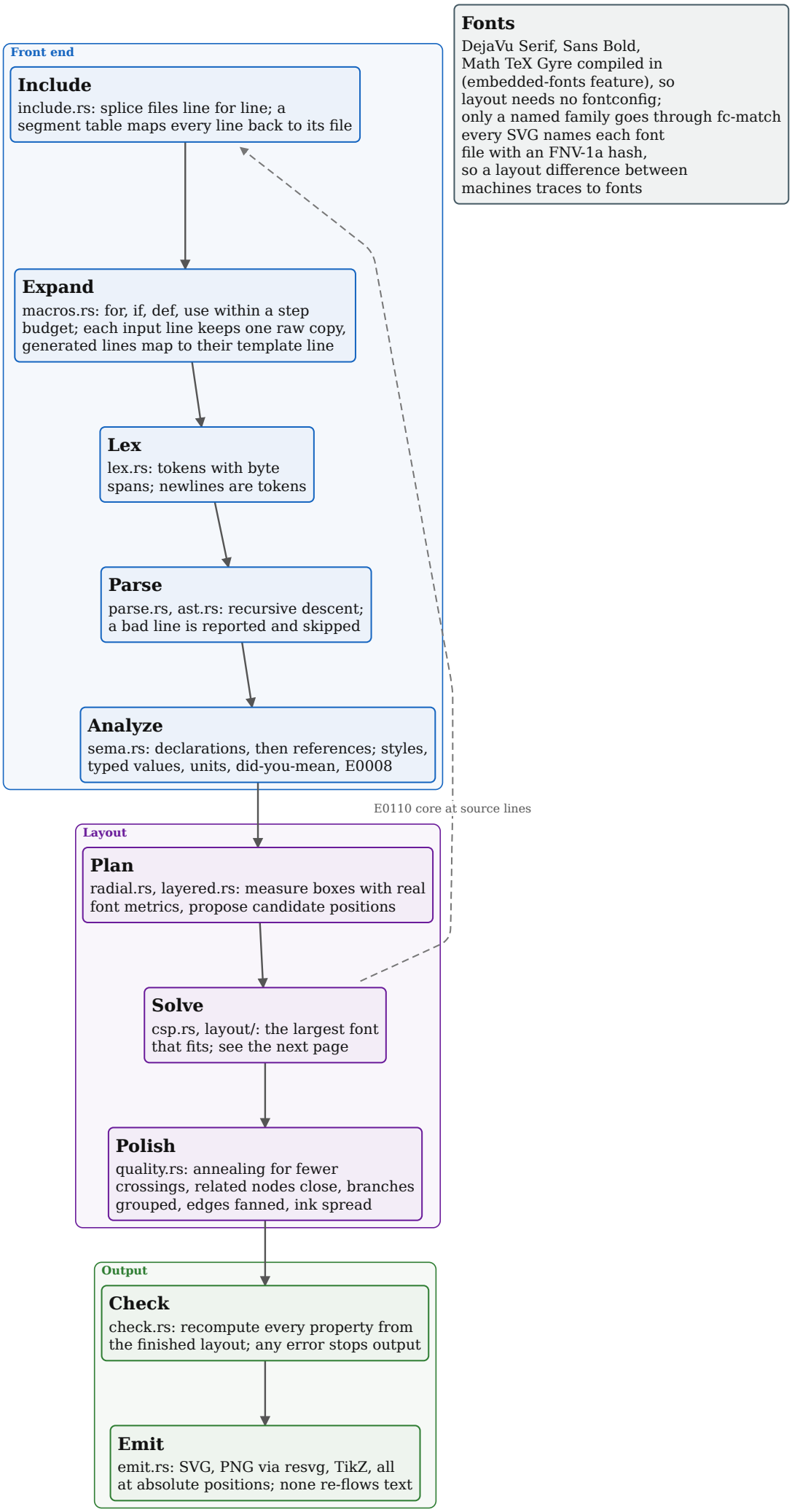
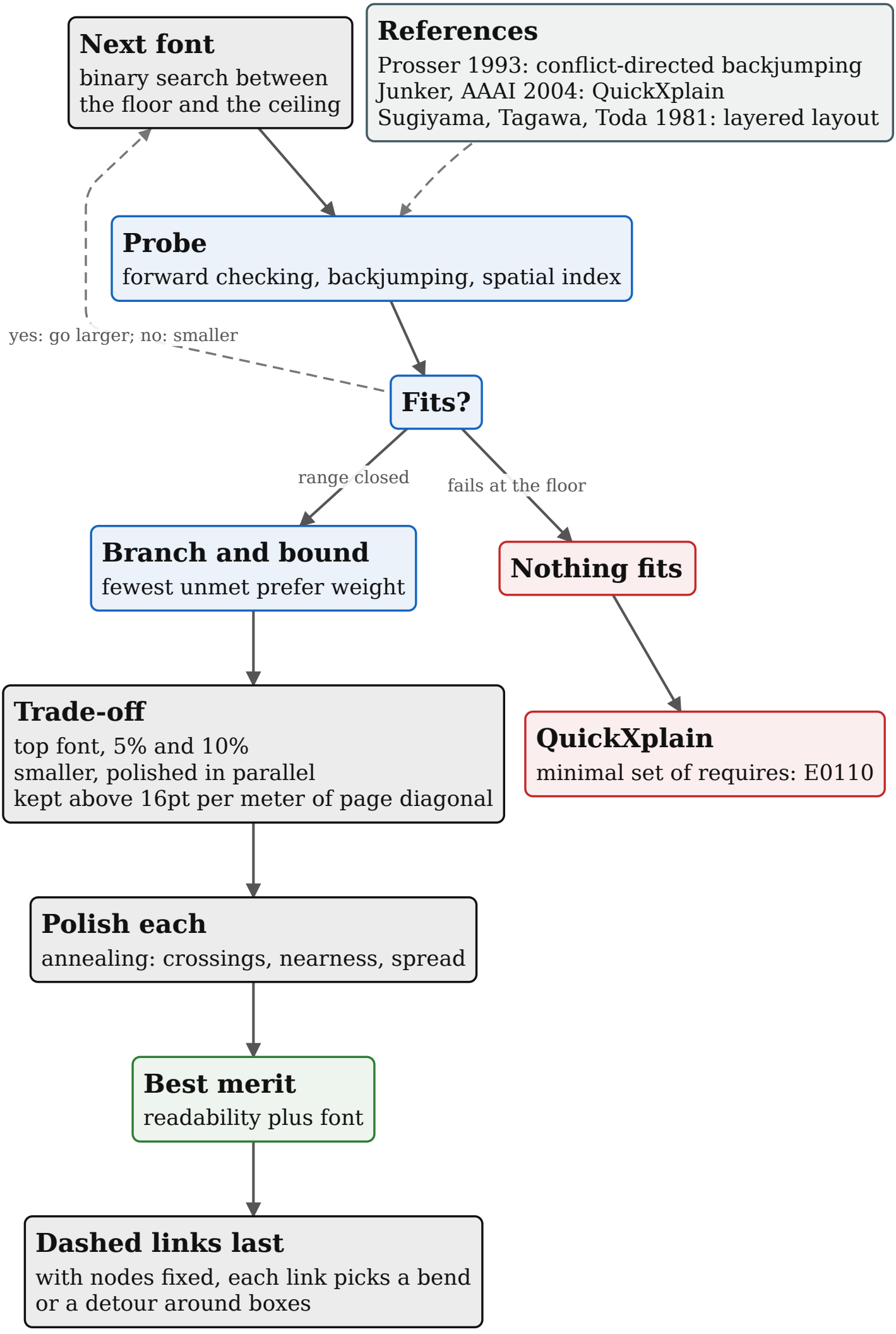
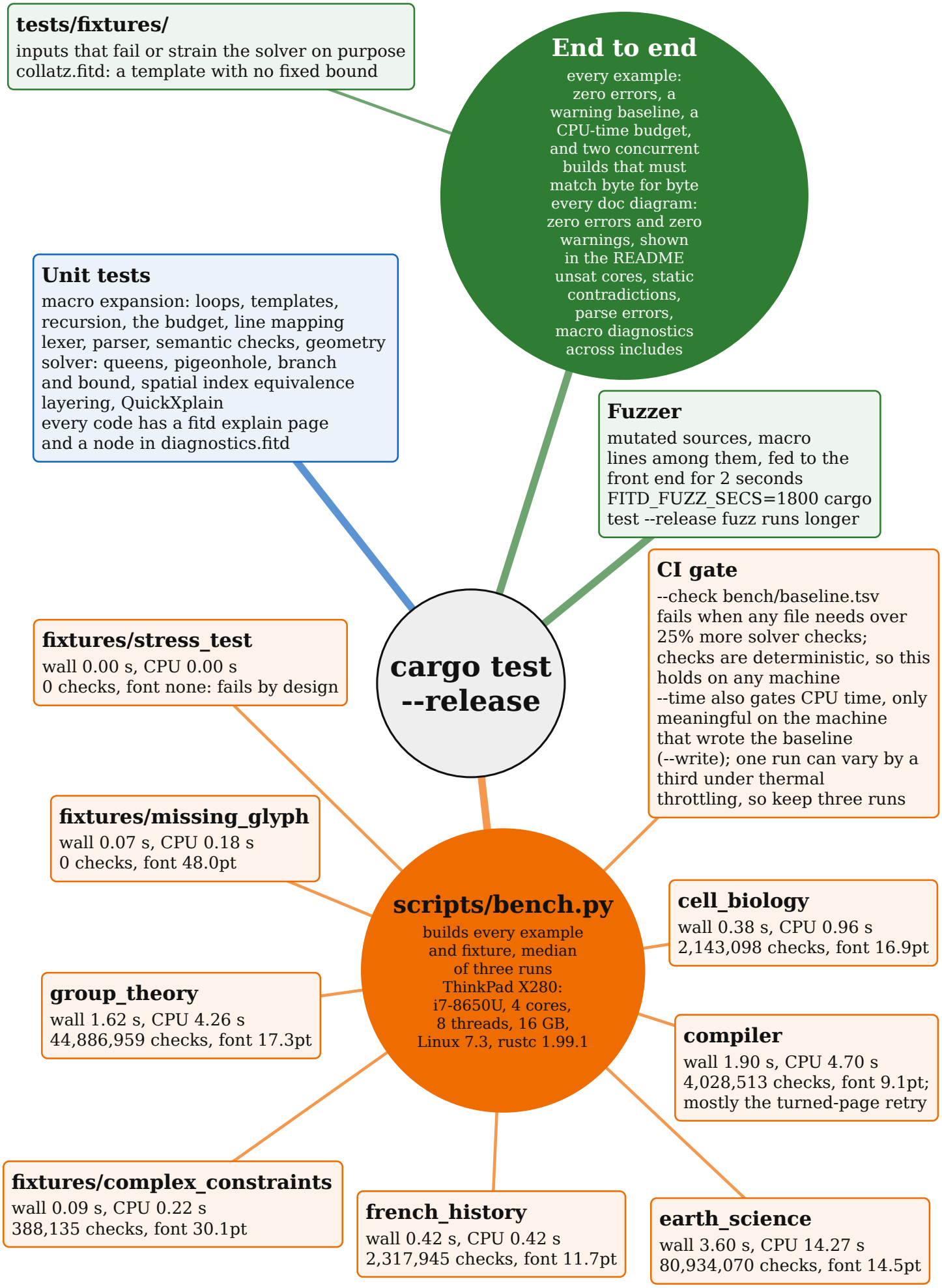

Fonts
DejaVu Serif, Sans Bold,
Math TeX Gyre compiled in
(embedded-fonts feature), so
layout needs no fontconfig;
only a named family goes through fc-match
every SVG names each font
file with an FNV-1a hash,
so a layout difference between
machines traces to fonts
Front end
Include
include.rs: splice files line for line; a
segment table maps every line back to its file
Expand
macros.rs: for, if, def, use within a step
budget; each input line keeps one raw copy,
generated lines map to their template line
Lex
lex.rs: tokens with byte
spans; newlines are tokens
Parse
parse.rs, ast.rs: recursive descent;
a bad line is reported and skipped
Analyze
sema.rs: declarations, then references; styles,
typed values, units, did-you-mean, E0008
E0110 core at source lines
Layout
Plan
radial.rs, layered.rs: measure boxes with real
font metrics, propose candidate positions
Solve
csp.rs, layout/: the largest font
that fits; see the next page
Polish
quality.rs: annealing for fewer
crossings, related nodes close, branches
grouped, edges fanned, ink spread
Output
Check
check.rs: recompute every property from
the finished layout; any error stops output
Emit
emit.rs: SVG, PNG via resvg, TikZ, all
at absolute positions; none re-flows text


Next font
binary search between the floor and the ceiling
References
Prosser 1993: conflict-directed backjumping
Junker, AAAI 2004: QuickXplain
Sugiyama, Tagawa, Toda 1981: layered layout
Probe
forward checking, backjumping, spatial index
yes: go larger; no: smaller
Fits?
range closed
fails at the floor
Branch and bound
fewest unmet prefer weight
Nothing fits
Trade-off
top font, 5% and 10%
smaller, polished in parallel
kept above 16pt per meter of page diagonal
QuickXplain
minimal set of requires: E0110
Polish each
annealing: crossings, nearness, spread
Best merit
readability plus font
Dashed links last
with nodes fixed, each link picks a bend
or a detour around boxes


tests/fixtures/
inputs that fail or strain the solver on purpose
collatz.fitd: a template with no fixed bound
End to end
every example: zero errors, a warning baseline, a CPU-time budget, and two concurrent builds that must match byte for byte
every doc diagram: zero errors and zero warnings, shown in the README
unsat cores, static contradictions, parse errors, macro diagnostics across includes
Unit tests
macro expansion: loops, templates, recursion, the budget, line mapping
lexer, parser, semantic checks, geometry
solver: queens, pigeonhole, branch and bound, spatial index equivalence
layering, QuickXplain
every code has a fitd explain page and a node in diagnostics.fitd
Fuzzer
mutated sources, macro lines among them, fed to the front end for 2 seconds
FITD_FUZZ_SECS=1800 cargo test --release fuzz runs longer
CI gate
--check bench/baseline.tsv fails when any file needs over 25% more solver checks; checks are deterministic, so this holds on any machine
--time also gates CPU time, only meaningful on the machine that wrote the baseline (--write); one run can vary by a third under thermal throttling, so keep three runs
cargo test --release
fixtures/stress_test
wall 0.00 s, CPU 0.00 s
0 checks, font none: fails by design
fixtures/missing_glyph
wall 0.07 s, CPU 0.18 s
0 checks, font 48.0pt
scripts/bench.py
builds every example and fixture, median of three runs
ThinkPad X280: i7-8650U, 4 cores, 8 threads, 16 GB, Linux 7.3, rustc 1.99.1
cell_biology
wall 0.38 s, CPU 0.96 s
2,143,098 checks, font 16.9pt
group_theory
wall 1.62 s, CPU 4.26 s
44,886,959 checks, font 17.3pt
compiler
wall 1.90 s, CPU 4.70 s
4,028,513 checks, font 9.1pt; mostly the turned-page retry
fixtures/complex_constraints
wall 0.09 s, CPU 0.22 s
388,135 checks, font 30.1pt
french_history
wall 0.42 s, CPU 0.42 s
2,317,945 checks, font 11.7pt
earth_science
wall 3.60 s, CPU 14.27 s
80,934,070 checks, font 14.5pt